Reviewer Pack — Minimal Order of p-Adic Valuation Groups and SAT Clause Subs…

Entry 4a7a5f98e73e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 06:48:27 UTC

Minimal Order of p-Adic Valuation Groups and SAT Clause Subset Complexity Inequality

Entry ID: `4a7a5f98e73e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 06:48:27 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

> For every satisfiable Boolean formula φ with n variables, the minimal order of a p-adic valuation group that can represent all possible assignments to φ is at most Θ(n log(p)).

Rationale (proposer's reasoning):

> p-adic analysis provides a framework for studying number-theoretic properties in a way that could potentially expose hidden structures within the satisfiability problem. The conjecture links the complexity of representing all clause subsets with a p-adic valuation group, which has applications in cryptography and other areas where p-adic numbers are relevant.

Taxonomy category: `p-adic_analysis` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `62c0fe6b0c5c7eba`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Boolean formula φ with n variables, if the minimal order of a p-adic valuation group G that can represent all assignments is O(n log(p)) for all 30 seeds, then φ supports the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `p-adic analysis AND SAT clause subset complexity` - `valuation groups in p-adic analysis AND Boolean satisfiability` - `θ(n log(p)) AND minimal order of p-adic valuation groups AND SAT`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/2201.08874v2] Note on p-adic Local Functional Equation - [http://arxiv.org/abs/1701.07658v2] On unitarity of some representatations of classical p-adic groups I - [http://arxiv.org/abs/1701.07662v2] On unitarity of some representations of classical p-adic groups II - [http://arxiv.org/abs/1709.00630v2] Unitarizability in generalized rank three for classical p-adic groups - [http://arxiv.org/abs/1406.6311v4] The Physics of the B Factories - [http://arxiv.org/abs/2308.12940v4] Measurement of the Z boson production cross-section in pp collisions at $\sqrt{s} = 5.02$ TeV

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_sat_formula(n):
        clauses = []
        for _ in range(2 ** n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for i in range(n) for j in range(i + 1, n)):
                clauses.append(clause)
        return clauses

    def p_adic_valuation_group_size(n, p):
        return math.ceil(n * math.log(p))

    n_max = 0
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
```

```
        if n > 40:
            break

        for _ in range(5):
            clauses = generate_random_sat_formula(n)
            p = random.randint(2, 100)
            metric_value = p_adic_valuation_group_size(n, p)

            if instances_tested == 0:
                n_max = n

            total_metric_value += metric_value
            instances_tested += 1

            if not conjecture_holds and counterexample == "":
                continue

            expected_value = n * math.log(p)
            if abs(metric_value - expected_value) > 1e-6:
                conjecture_holds = False
                counterexample = f"n={n}, p={p}, metric_value={metric_value}, expected_value={expe

    return {
        "metric_name": "p-adic valuation group size",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **result}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"{results[sum(1 for r in results if not r['conje
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11502
2	preregistration	ollama_remote	glm4:latest	0	0	9162
3	novelty	ollama_remote	glm4:latest	0	0	8014
4	novelty	ollama_remote	glm4:latest	0	0	11329
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14780
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7985
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6601
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35996
9	judge	ollama_remote	glm4:latest	0	0	56071

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 161441 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4a7a5f98e73e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4a7a5f98e73e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4a7a5f98e73e.tar.gz` (if generated)